Jr.Pen 16. Python

1. Python: Simple Demo

Task 1 - Introduction

Python is a high-level general-purpose programming language. By *high-level*, we mean it hides most implementation details; *general-purpose* means you can use Python for a wide variety of scenarios, from web applications and automation scripts to data science and machine learning.

```
ubuntu@tryhackme:~$ python guess_the_number.py
I'm thinking of a number between 1 and 20
Take a guess: 10
Too high, try again.
Take a guess: 5
Too low, try again.
Take a guess: 7
Too low, try again.
Take a guess: 8
You got it in 4 tries!
```

Task 2 - Variables

Python offers the `random.randint()` method, which returns a random integer within the specified bounds. For example, `random.randint(1, 20)` returns a random number between 1 and 20. Since the user has not made any attempts, we set `tries` to 0; moreover, we set `guess` to a value outside the valid range. To tell the user that the program picked a number, we use Python's `print()` method to display an informational message on the screen. We show the current Python script below:

```
import random  # gives us tools for picking random numbers

secret = random.randint(1, 20)  # a <= secret <= b
tries = 0
guess = 0  # start with a value that cannot be the secret (since secret is 1..20)

print("I'm thinking of a number between 1 and 20")
```

This code snippet achieves the following:

1. Use the `random.randint()` method from the `random` library to pick a random number between 1 and 20 and save it to `secret`.
2. Creates two variables, `tries` and `guess`, and sets them to `0`.
3. Display a message on the screen to inform the user that a number has been picked.

With our variables ready, we prompt the user to make a guess and set `guess` accordingly. We begin by saving the user's input to a temporary `text` variable, then use Python's `int()` to convert it to an integer. Furthermore, we need to increment `tries` each time the user makes a guess.

```
text = input("Take a guess: ")  # input() returns text (a string)
guess = int(text)  # convert the text to a number

tries = tries + 1  # add 1 try (written long-form for clarity)
```

So far, our program does the following:

1. Pick a random number between `1` and `20` and save it in `secret`.
2. Ask the user to take a guess, convert it to an integer, and save it in `guess`.
3. Increment `tries` by 1.

The Incomplete Draft: In the next task, we upgrade this program so that it gives the user some feedback on how their `guess` compares to the `secret` number. The program below is saved as `guess_v1.py` in the `/home/ubuntu/Python-Demo` directory.

```
import random  # gives us tools for picking random numbers

secret = random.randint(1, 20)  # a <= secret <= b
tries = 0
guess = 0  # start with a value that cannot be the secret (since secret is 1..20)

print("I'm thinking of a number between 1 and 20")

text = input("Take a guess: ")  # input() returns text (a string)
guess = int(text)  # convert the text to a number

tries = tries + 1  # add 1 try
```

Answer the questions below

> What is the name of the function we used to display text on the screen?

print()

> What is the name of the function that we used to convert user input to an integer?

int()

Task 3 - Conditional Statements

it is only logical that if the guess is not less than or larger than the secret number, it would be equal to the secret number. In programming, whenever the condition after the "**if**" is not true (written as `if CONDITION:` in Python), the program checks the condition after the "**else if**" (written as `elif CONDITION:` in Python), and if it is also false, it checks the next "else if". If there are no more "else if" statements left, the program executes the final "**else**" (written as `else:` in Python). Let's convert the above pseudo-code to working Python code.

```
# Give a hint using if / elif / else.
if guess < 1 or guess > 20:
    print("That number is out of range. Try again.")
elif guess < secret:
```

```
    print("Too low, try again.")
elif guess > secret:
    print("Too high, try again.")
else:
    print("You got it in", tries, "tries!")
```

In this implementation, comparing the user's `guess` to the chosen `secret` will lead to various cases:

- The user makes a `guess` outside the allowed bounds, i.e., 1 and 20
- The user's `guess` is less than `secret`
- The user's `guess` is greater than `secret`
- The user's `guess` is correct

If the above looks confusing, consider the following numerical examples.

Numerical Example 1: We assume that the secret number is 10. If the user inputs 30, the first `if` condition will evaluate to true, and Python will execute the part after `if guess < 1 or guess > 20:` and print "That number is out of range. Try again."

Numerical Example 2: We assume that the secret number is 10. Suppose the user inputs 5, the first `if` condition will evaluate to false because 5 is neither less than 1 nor greater than 20. Python will check the condition following the first `elif` and it will evaluate to true because 5 is indeed less than 10. Consequently, the program will print "Too low, try again."

Numerical Example 3: We assume that the secret number is 10. Suppose the user inputs 15, the first `if` condition will evaluate to false because 15 is neither less than 1 nor greater than 20. The program will check the condition following the first `elif` and it will also evaluate to false because 15 is not less than 10. The program will check the condition following the second `elif`, and this one will evaluate to true, because 15 is larger than 10. As a result, the program will display, "Too high, try again."

The First Draft: In this task, we built a working program that you can test for yourself and edit in the attached VM. The main limitation is that this program gives the user one chance. In the next task, we upgrade this program to allow the user an infinite number of tries to make the correct guess. The program below is saved as `guess_v2.py` in the `/home/ubuntu/Python-Demo` directory.

```
import random  # gives us tools for picking random numbers

secret = random.randint(1, 20)  # a <= secret <= b
tries = 0
guess = 0  # start with a value that cannot be the secret (since secret is 1..20)

print("I'm thinking of a number between 1 and 20")

text = input("Take a guess: ")  # input() returns text (a string)
guess = int(text)  # convert the text to a number

tries = tries + 1  # add 1 try

# Give a hint using if / elif / else.
if guess < 1 or guess > 20:
    print("That number is out of range. Try again.")
```

```
elif guess < secret:
    print("Too low, try again.")
elif guess > secret:
    print("Too high, try again.")
else:
    print("You got it in", tries, "tries!")
```

Answer the questions below

> How does Python write “else if”?

elif

> What will the program display if the user’s input is 50?

That number is out of range. Try again.

Task 4 - Iterations

In Python, “does not equal” is written as `!=`. In other words, the condition “`guess` does not equal `secret`” is written as `guess != secret`. To write this as a `while` loop, we use the form `while CONDITION:`; in our case, it is `while guess != secret:`.

```
# Repeat until the user guesses the secret number.
while guess != secret:
    text = input("Take a guess: ")  # input() returns text (a string)
    guess = int(text)  # convert the text to a number

    tries = tries + 1  # add 1 try

    # Give a hint using if / elif / else.
    if guess < 1 or guess > 20:
        print("That number is out of range. Try again.")
    elif guess < secret:
        print("Too low, try again.")
    elif guess > secret:
        print("Too high, try again.")
    else:
        print("You got it in", tries, "tries!")
```

The code above will execute as follows. First, Python will check the condition after the `while`; if it is true, it will run all the indented lines. If the condition is false, it terminates.

Numerical Example 1: We assume that the secret number is 10. In the first run, the guess is 0.
The `while` condition will evaluate to true because 10 is not equal to 0. The program will ask the user to “Take a guess,” and it will proceed from there.

Numerical Example 2: We assume that the secret number is 10. If the user entered 5 earlier, it means they did not guess the number in the previous attempt. The `while` condition will evaluate to true because 10 is not equal to 5. The program will give the user another chance and prompt them to “Take a guess.”

Numerical Example 3: We assume that the secret number is 10. If the user entered 10 earlier, it means they successfully guessed the number in the previous attempt. Let's see if the program will prompt them for another guess. The `while` loop will evaluate to false because 10 is equal to 10; in other words, `10 != 10` is false. The indented code block within the `while` statement will no longer execute.

The Second Draft: In this task, we have completed coding our game. You can test it for yourself and edit in the attached VM. The program below is saved as `guess_v3.py` in the `/home/ubuntu/Python-Demo` directory.

```
import random  # gives us tools for picking random numbers

# ----------------------------
# Guess the Number (Beginner Demo)
# ----------------------------
# The computer picks a secret number.
# The player keeps guessing until they find it.

secret = random.randint(1, 20)  # a <= secret <= b
tries = 0
guess = 0  # start with a value that cannot be the secret (since secret is 1..20)

print("I'm thinking of a number between 1 and 20")

# Repeat until the user guesses the secret number.
while guess != secret:
    text = input("Take a guess: ")  # input() returns text (a string)
    guess = int(text)  # convert the text to a number

    tries = tries + 1  # add 1 try

    # Give a hint using if / elif / else.
    if guess < 1 or guess > 20:
        print("That number is out of range. Try again.")
    elif guess < secret:
        print("Too low, try again.")
    elif guess > secret:
        print("Too high, try again.")
    else:
        print("You got it in", tries, "tries!")
```

Answer the questions below

> What type of loop does this program use?

while

> What will the program display if the user makes the correct guess in 3 tries?

You got it in 3 tries!

Task 5 - Conclusion

In this room, we covered three key pillars in imperative programming languages:

- Variables
- Conditionals with `if` and `else`
- Loops with `while`

If this is your first exposure to Python, don't worry about being able to write a similar program yourself. Learning programming languages requires practice, and although you can now understand and explain this program, practice is indispensable to write something similar. We strongly encourage you to compare the code in this room with that of the JavaScript: Simple Demo room (coming soon); this will give you a good perspective on how different modern programming languages are designed and written.

2. Python: Core Concepts

Task 1 - Introduction

You need to know how to work with **strings**, store data in **lists** and **dictionaries**, use a richer set of **operators**, and master a second type of loop: the `for` **loop**. That is exactly what this room covers.

Learning Objectives

- Review Python variables, conditional statements, and loops (covered in Simple Demo)
- Work with Python data types and perform type conversions
- Use formatted strings (f-strings) and augmented assignment operators
- Manipulate strings using built-in methods
- Store and retrieve data with lists and dictionaries
- Use comparison, logical, arithmetic, and membership operators
- Iterate over data using `for` loops, `while` loops, and `range()`
- Control loop flow with `break` and `continue`

Task 2 - Quick Review: Hello World, Variables, and Conditionals

The simplest Python program outputs text to the screen using the `print()` function. In the Python: Simple Demo room, we used `print()` to tell the user that the computer had picked a secret number. Let's revisit the basics.

```
# This is a comment. Python ignores lines starting with #.
print("Hello World")
```

When you run this, the text `Hello World` appears on the screen. A few things to note:

- Lines starting with `#` are **comments**. The computer ignores them; they exist to help *humans* understand the code.
- The `print()` function outputs whatever is placed between its parentheses.
- Text (called a **string**) must be enclosed in quotation marks, either `"double"` or `'single'`.

Open the file `hello.py` in VS Code on the attached VM, run it, and confirm you see `Hello World` in the terminal.

Variables and Data Types: Variables let you store and update data. As we saw in Simple Demo, we stored the secret number in a variable called `secret` and the user's input in `guess`:

```
food = "ice cream"   # a string (text)
money = 2000          # an integer (whole number)
```

In the example above, the variable `food` stores the string `"ice cream"`, while `money` stores the integer `2000`. Variables are powerful because you can change their values throughout your program:

```
age = 30
age = age + 1
print(age)   # 31
```

Every value in Python has a **data type**. The core types you need to know are:

Type	Name	Example	Description
`str`	String	`"hello"`	Text, characters, symbols
`int`	Integer	`42`	Whole numbers
`float`	Float	`3.14`	Numbers with decimal points
`bool`	Boolean	`True` / `False`	Logical on/off, yes/no values
`list`	List	`[1, 2, 3]`	Ordered collection of items

You can check a value's type using `type()`:

```
username = "admin"
port = 8080
print(type(username))   # <class 'str'>
print(type(port))       # <class 'int'>
```

Open `types_demo.py` on the VM and run it. Observe the output and try changing the variable values to see how `type()` responds.

Conditional Statements and Logical Operators: In Simple Demo, we used `if`, `elif`, and `else` to compare the user's guess to the secret number. Let's recap the structure:

```
age = 18
if age < 17:
    print("You are NOT old enough to drive")
else:
    print("You are old enough to drive")
```

The key components are:

- The `if` keyword followed by a **condition** and a colon `:`
- An indented code block that runs only when the condition is `True`
- An optional `else:` block that runs when the condition is `False`
- One or more `elif CONDITION:` blocks for additional checks (as we used in our guessing game)

Comparison operators let you build conditions:

Operator	Meaning	Example
`==`	Equal to	`x == 5`
`!=`	Not equal to	`x != 5`
`<`	Less than	`x < 5`
`>`	Greater than	`x > 5`
`<=`	Less than or equal to	`x <= 5`
`>=`	Greater than or equal to	`x >= 5`

Logical operators combine multiple conditions:

Operator	Meaning	Example
`and`	True only if *both* sides are true	`x > 0 and x < 100`
`or`	True if *at least one* side is true	`x == 1 or x == 10`
`not`	Inverts the boolean value	`not is_locked`

Consider the following example:

```
name = "bob"
hungry = True

if name == "bob" and hungry == True:
    print("Bob is hungry")
elif name == "bob" and not hungry:
    print("Bob is not hungry")
else:
    print("Not sure who this is or if they are hungry")
```

A critical distinction worth repeating: `=` is the **assignment** operator (it stores a value in a variable), while `==` is the **equality** operator (it compares two values). Confusing the two is one of the most common beginner mistakes.

What's New: Type Conversion and f-strings

If you completed Simple Demo, the material above should feel like familiar territory. The rest of this room introduces concepts that go well beyond what we covered in the guessing game. Let's start with two improvements to how we work with variables.

Type Conversion: As we saw in Simple Demo, `input()` always returns a string. When the user typed `10` at the prompt, Python stored it as the text `"10"`, not the number `10`. We used `int()` to convert it. Python provides several built-in conversion functions:

Function	Converts To	Example
`int()`	Integer	`int("42")` gives `42`
`float()`	Float	`float("3.14")` gives `3.14`
`str()`	String	`str(42)` gives `"42"`
`bool()`	Boolean	`bool(0)` gives `False`

```
text = input("Enter a port number: ")    # user types 443
print(type(text))                         # <class 'str'>
```

```
port = int(text)                        # convert to integer
print(type(port))                       # <class 'int'>
print(port + 1)                         # 444
```

Without the `int()` conversion, `text + 1` would cause an error because Python does not know how to add a number to a piece of text.

Formatted Strings (f-strings): In Simple Demo, we used commas inside `print()` to display variables alongside text, like `print("You got it in", tries, "tries!")`. This approach works, but modern Python offers a cleaner alternative called **f-strings** (formatted string literals). An f-string starts with the letter `f` before the opening quotation mark and lets you embed variables directly inside curly braces:

```
username = "admin"
port = 443

# Old approach (comma-separated, as used in Simple Demo)
print("User", username, "is on port", port)

# Modern approach (f-string)
print(f"User{username} is on port{port}")
```

Both lines produce the same output: `User admin is on port 443`. The f-string version is easier to read and gives you precise control over spacing and formatting. We will use f-strings throughout the rest of this room.

Augmented Assignment Operators: In Simple Demo, we wrote `tries = tries + 1` to increment a counter. Python provides a shorthand for this pattern called **augmented assignment**:

```
count = 0
count += 1    # same as count = count + 1
count -= 1    # same as count = count - 1
count *= 2    # same as count = count * 2
count /= 4    # same as count = count / 4
```

These shortcuts make code shorter and reduce the chance of typos. You will see `+=` used frequently in loops and counters. Open `fstrings_demo.py` on the attached VM, run it, and observe how f-strings and augmented assignment operators work in practice.

Answer the questions below

> What built-in function reveals the data type of a value?

type()

> If a user types `3.14` at an `input()` prompt, what data type does Python store it as before any conversion?

str

```
ubuntu@tryhackme:~/Core-Concepts$ python types_demo.py
username = admin
  type: <class 'str'>

port = 8080
  type: <class 'int'>

temperature = 36.6
  type: <class 'float'>
```

> On the attached VM, open and run `fstrings_demo.py`. What is the last line printed to the terminal?

Scan complete: 192.168.1.1 has 3 open ports

```
ubuntu@tryhackme:~/Core-Concepts$ python fstrings_demo.py
Old style: admin is on port 443
f-string:  admin is on port 443
Total cost: $149.97

Starting count: 0
After += 5:    5
After -= 2:    3
After *= 4:    12
After //= 3:   4

Scan complete: 192.168.1.1 has 3 open ports
```

Task 3 - Working with Strings

In Python, that inspection is powered by **string methods**, and mastering them is essential for any script that processes text. In security, that means nearly every script you will ever write.

A **string** is a sequence of characters enclosed in quotes. You can use single quotes (`'hello'`), double quotes (`"hello"`), or triple quotes (`"""hello"""`) for multi-line text. Python treats them all as the same data type.

String Length: The `len()` function returns how many characters a string contains. This is a fundamental check in many security scripts: is the password long enough? Is the input within expected bounds?

```
password = "Tr0ub4dor"
length = len(password)
print(f"Password length:{length}")   # Password length: 9
```

String Indexing and Slicing: Each character in a string occupies a numbered position called an **index**. Python indexes start at `0`, not `1`. Consider the string `"Python"`:

Index	0	1	2	3	4	5
Character	P	y	t	h	o	n

```
word = "Python"
print(word[0])     # P   (first character)
print(word[5])     # n   (last character)
print(word[-1])    # n   (negative index counts from the end)
```

Slicing lets you extract a portion of a string using the syntax `string[start:end]`. The `start` index is included; the `end` index is excluded.

```
word = "Python"
print(word[0:3])   # Pyt   (characters at index 0, 1, 2)
```

```
print(word[2:])    # thon  (from index 2 to the end)
print(word[:4])    # Pyth  (from the start to index 3)
```

Useful String Methods: Python strings come with dozens of built-in methods. Here are the ones you will reach for most often:

Method	What It Does	Example	Result
`.upper()`	Converts to uppercase	`"hello".upper()`	`"HELLO"`
`.lower()`	Converts to lowercase	`"HELLO".lower()`	`"hello"`
`.strip()`	Removes leading/trailing whitespace	`" hi ".strip()`	`"hi"`
`.replace(a, b)`	Replaces all occurrences of `a` with `b`	`"cat".replace("c", "b")`	`"bat"`
`.split(sep)`	Splits into a list at each `sep`	`"a,b,c".split(",")`	`["a", "b", "c"]`
`.startswith(x)`	Checks if string begins with `x`	`"http://".startswith("http")`	`True`
`.endswith(x)`	Checks if string ends with `x`	`"file.txt".endswith(".txt")`	`True`
`.count(x)`	Counts occurrences of `x`	`"banana".count("a")`	`3`

Character Checks: For tasks like password validation, we need to know whether a character is a digit, an uppercase letter, or something else. Python provides methods for exactly this:

```
char = "A"
print(char.isupper())    # True
print(char.islower())    # False
print(char.isdigit())    # False
print(char.isalpha())    # True   (is it a letter?)
print(char.isalnum())    # True   (is it a letter or digit?)
```

These methods return a boolean (`True` or `False`), which makes them perfect for use inside `if` statements. We will combine them with loops later in this room to scan every character in a string.

The `in` Operator: Python's `in` operator checks whether a substring exists within a larger string. It returns `True` or `False`:

```
url = "https://tryhackme.com/room/pythoncoreconcepts"

print("tryhackme" in url)     # True
print("hackthebox" in url)    # False
print("https" in url)         # True
```

The `in` operator is one of the most versatile tools in Python; we will see it again with lists and dictionaries in the next task.

Open `strings_demo.py` on the VM. The script contains several string operations. Run it and study its output. Then modify the `test_string` variable at the top to your own name and rerun to see how the output changes.

Answer the questions below

> What function returns the number of characters in a string?

len()

> Given `word = "TryHackMe"`, what does `word[3:7]` return?

Hack

> What string method converts `"ADMIN"` to `"admin"`?

.lower()

Task 4 - Lists and Dictionaries

Imagine you are responsible for monitoring a network. You have a list of IP addresses to scan, a collection of open ports for each host, and a mapping of port numbers to service names (port 22 is SSH, port 80 is HTTP, and so on). Storing each of these in a separate variable would be impractical. Python gives us two powerful **data structures** to handle collections of data: **lists** and **dictionaries**.

Lists: A **list** is an ordered collection of items enclosed in square brackets. Items can be of any data type, and the list preserves their order.

```
ports = [22, 80, 443, 8080]
usernames = ["admin", "root", "guest"]
mixed = ["server1", 443, True]
```

Accessing and Modifying List Elements: Lists use the same indexing system as strings. The first element is at index `0`:

```
ports = [22, 80, 443, 8080]

print(ports[0])      # 22
print(ports[-1])     # 8080   (last element)
print(ports[1:3])    # [80, 443]   (slicing works on lists too)
```

You can change an element by assigning a new value to its index:

```
ports[0] = 2222
print(ports)         # [2222, 80, 443, 8080]
```

Common List Methods

Method	What It Does	Example
`.append(x)`	Adds `x` to the end	`ports.append(3306)`
`.remove(x)`	Removes the first occurrence of `x`	`ports.remove(80)`
`.pop(i)`	Removes and returns the item at index `i`	`ports.pop(0)`
`.sort()`	Sorts the list in ascending order	`ports.sort()`
`.reverse()`	Reverses the list order	`ports.reverse()`
`len(list)`	Returns the number of items	`len(ports)`

The `in` operator, which we introduced with strings, works on lists as well:

```
common_passwords = ["123456", "password", "admin", "letmein"]
```

```
if "password" in common_passwords:
    print("This password is in the common list.")
```

Dictionaries: A **dictionary** stores data as **key-value pairs** enclosed in curly braces. One analogy would be a physical dictionary: you look up a *word* (the key) to find its *definition* (the value). In technical terms, a dictionary lets you associate one piece of data with another for quick retrieval.

```
services = {
    22: "SSH",
    80: "HTTP",
    443: "HTTPS",
    3306: "MySQL"
}
```

Accessing Dictionary Values: You retrieve a value by passing its key inside square brackets:

```
print(services[22])     # SSH
print(services[443])    # HTTPS
```

Adding, Updating, and Removing Entries

```
# Add a new entry
services[8080] = "HTTP-Alt"

# Update an existing entry
services[22] = "OpenSSH"

# Remove an entry
del services[3306]

print(services)
# {22: 'OpenSSH', 80: 'HTTP', 443: 'HTTPS', 8080: 'HTTP-Alt'}
```

Checking for Keys

```
if 22 in services:
    print(f"Port 22 runs{services[22]}")
```

Useful Dictionary Methods

Method	Returns
`.keys()`	All keys in the dictionary
`.values()`	All values in the dictionary
`.items()`	All key-value pairs as tuples
`.get(key, default)`	Value for `key`, or `default` if the key does not exist

The `.get()` method is particularly useful because accessing a nonexistent key with square brackets causes an error, while `.get()` lets you supply a safe fallback:

```
result = services.get(9999, "Unknown")
print(result)   # Unknown
```

Answer the questions below

> What method adds an element to the end of a list?

.append()

> Given `services = {22: "SSH", 80: "HTTP"}`, what does `services[80]` return?

HTTP

> What dictionary method lets you retrieve a value with a safe fallback if the key does not exist?

.get()

Task 5 - Arithmetic and Membership Operators

In Simple Demo, we used comparison operators like `<` and `>` to compare the user's guess to the secret number. In this task, we expand our operator toolkit with new arithmetic operators and a closer look at how Python evaluates compound conditions.

New Arithmetic Operators: You already know addition (`+`), subtraction (`-`), multiplication (`*`), and division (`/`). Let's add three more to the set:

Operator	Name	Example	Result
`**`	Exponent	`2 ** 8`	`256`
`//`	Floor Division	`7 // 2`	`3`
`%`	Modulus (remainder)	`7 % 2`	`1`

The **modulus** operator (`%`) returns the remainder after division. It is commonly used to check whether a number is even or odd: if `number % 2 == 0`, the number is even.

Floor division (`//`) divides and then rounds down to the nearest whole number. Regular division (`/`) always returns a float: `7 / 2` gives `3.5`, while `7 // 2` gives `3`.

The **exponent** operator (`**`) raises the left number to the power of the right number. For example, `2 ** 8` calculates 2 raised to the 8th power, which is `256`. In security, you will often see this when discussing key sizes: a 128-bit key has `2 ** 128` possible combinations.

The Membership Operator: `in` It deserves dedicated attention because it works with almost every collection type in Python and will play a central role in many security scripts.

```
# Checking if a password is in a list of common passwords
common = ["123456", "password", "qwerty", "letmein"]
user_password = "qwerty"

if user_password in common:
    print("This password is too common.")
```

You can negate it with `not in`:

```
if user_password not in common:
    print("Good. This password is not in the common list.")
```

Combining Operators in Practice: Consider a practical example. A password might be considered moderate if it is at least 8 characters long *and* contains at least one digit:

```
password = "Tr0ubador"
length = len(password)
has_digit = any(char.isdigit() for char in password)

if length >= 8 and has_digit:
    print("Moderate strength")
elif length >= 8 or has_digit:
    print("Weak, but has some merit")
else:
    print("Very weak")
```

Don't worry if the `any()` line looks unfamiliar; we will revisit it in the companion room when we cover functions. For now, focus on how `and` requires both conditions to be true, while `or` requires only one.

Open `operators_demo.py` on the attached VM. The script walks you through each operator with printed examples. Run it and observe the output.

Answer the questions below

> What operator returns the remainder of a division?

%

> What does `10 // 3` evaluate to?

3

> What does `2 ** 10` evaluate to?

1024

Task 6 - Loops: for and while

In Simple Demo, we used a `while` loop to keep asking the user for guesses until they found the secret number. The `while` loop repeats as long as a condition is true; it is perfect when the number of iterations depends on some runtime condition. Python offers a second type of loop, the `for` **loop**, which is designed for iterating over a sequence of items. Let's explore both.

Quick Recap: `while` **Loops:** A `while` loop runs as long as its condition evaluates to `True`:

```
attempts = 0
max_attempts = 3

while attempts < max_attempts:
    password = input("Enter password: ")
```

```
    attempts += 1
    print(f"Attempt{attempts} of{max_attempts}")
```

This loop asks for a password up to 3 times. Once `attempts` reaches 3, the condition `attempts < max_attempts` becomes `False`, and the loop stops.

The `for` Loop: Consider the case where you have a list of IP addresses and you want to process each one. You know exactly how many items there are, and you want to visit each one in order. This is the sweet spot for a `for` loop.

```
targets = ["192.168.1.1", "192.168.1.2", "192.168.1.3"]

for ip in targets:
    print(f"Scanning{ip}...")
```

```
# The output would be:
Scanning 192.168.1.1...
Scanning 192.168.1.2...
Scanning 192.168.1.3...
```

The variable `ip` takes on the value of each element in the `targets` list, one at a time. On the first iteration, `ip` is `"192.168.1.1"`; on the second, it is `"192.168.1.2"`; and so on. The loop ends when every element has been visited. A string is also a sequence, so you can loop through it character by character. This is exactly what we need for tasks like password validation:

```
password = "S3cure!"

for char in password:
    if char.isdigit():
        print(f"Found digit:{char}")
    elif char.isupper():
        print(f"Found uppercase:{char}")
```

```
# The output would be:
Found uppercase: S
Found digit: 3
```

The `range()` Function: Sometimes you need a loop that runs a specific number of times without a predefined list. The `range()` function generates a sequence of numbers:

```
# range(stop): 0 to stop-1
for i in range(5):
    print(i)       # prints 0, 1, 2, 3, 4

# range(start, stop): start to stop-1
for i in range(1, 6):
    print(i)       # prints 1, 2, 3, 4, 5
```

```
# range(start, stop, step): with custom increment
for i in range(0, 20, 5):
    print(i)       # prints 0, 5, 10, 15
```

In programming, counting often starts from 0. Consequently, `range(5)` produces five numbers: 0, 1, 2, 3, and 4.

Iterating Over Dictionaries: As we mentioned in Task 4, dictionaries have `.keys()`, `.values()`, and `.items()` methods. These become especially powerful inside `for` loops:

```
services = {22: "SSH", 80: "HTTP", 443: "HTTPS"}

# Loop through both keys and values
for port, name in services.items():
    print(f"Port{port} ={name}")
```

```
# The output would be:
Port 22 = SSH
Port 80 = HTTP
Port 443 = HTTPS
```

`break` **and** `continue` Two keywords give you fine-grained control inside a loop:

- `break` immediately exits the loop entirely.
- `continue` skips the rest of the current iteration and moves to the next one.

```
# Stop scanning as soon as we find port 443
for port in [22, 80, 443, 8080]:
    if port == 443:
        print(f"Port{port} found. Stopping scan.")
        break
    print(f"Checked port{port}")
```

```
# The output would be:
Checked port 22
Checked port 80
Port 443 found. Stopping scan.
```

Notice that `8080` was never checked because `break` terminated the loop as soon as we found `443`.

```
# Skip blank lines when processing a file
lines = ["admin", "", "root", "", "guest"]

for line in lines:
    if line == "":
        continue       # skip empty strings
    print(f"Processing:{line}")
```

```
# The output would be:
Processing: admin
Processing: root
Processing: guest
```

Choosing Between `for` and `while`

A general rule of thumb: use a `for` loop when you know the number of iterations in advance (a list to process, a range of numbers, a string to scan). Use a `while` loop when the number of iterations depends on a condition that changes during execution (e.g., waiting for correct input or retrying until a connection succeeds).

On the VM, open `loops_demo.py` and run it. The script demonstrates `for` loops, `while` loops, `range()`, `break`, and `continue` in sequence. Study the output and try adding a new loop that prints every even number from 0 to 20 using `range()` with a step of 2.

Answer the questions below

> What type of loop is best suited for iterating over each item in a list?

for

> What does `range(3)` produce?

0, 1, 2

> What keyword immediately exits a loop?

break

Task 7 - Conclusion

In this room, we covered the data and control flow foundations of Python:

- **Data types and type conversion**: understanding `str`, `int`, `float`, `bool` and converting between them with `int()`, `str()`, `float()`, and `bool()`
- **Formatted strings**: f-strings for clean, readable output
- **Strings**: indexing, slicing, and powerful built-in methods like `.split()`, `.strip()`, `.lower()`, `.isdigit()`, and `.isupper()`
- **Lists and dictionaries**: storing collections of data and retrieving them efficiently with indexing, `.append()`, `.get()`, and the `in` operator
- **Operators**: arithmetic (`+`, , , `/`, `//`, `%`, `*`), comparison (`==`, `!=`, `<`, `>`), logical (`and`, `or`, `not`), and the membership operator `in`
- **Loops**: `for` loops for sequences and ranges, `while` loops for condition-based repetition, plus `break` and `continue`

3. Python: Building Scripts

Task 1 - Introduction

In this room, we will learn how to *assemble* those building blocks into real programs. Consider the difference between knowing individual musical notes and being able to play a song. In Core Concepts, you learned the

notes. In this room, you will play the song. Specifically, we will cover four capabilities that separate a code snippet from a real script: **functions** for organizing and reusing logic, **error handling** for building resilience, **file I/O** for reading input and writing output, and **libraries** for leveraging code that others have already written. By the end, we will combine every concept from both rooms into a single, working **Password Strength Checker**: a tool that evaluates passwords against length requirements, character variety rules, and a common-passwords wordlist.

Learning Objectives: After completing this room, you will:

- Define and call functions with parameters, return values, and defaults
- Understand variable scope
- Handle runtime errors with `try` / `except`
- Read from and write to files using context managers (`with`)
- Import modules from the Python standard library
- Install third-party packages with `pip`
- Combine all concepts into a complete, security-relevant program

Task 2 - Functions

As programs grow larger, you will find yourself writing the same logic in multiple places. Consider our Password Strength Checker: we might want to check password strength in one part of the program and then again later when the user retries. Copying and pasting the same block of code is error-prone and tedious. This is precisely the problem that **functions** solve. One analogy would be a vending machine. You press a button (the **call**), optionally insert money (the **arguments**), and the machine gives you a drink (the **return value**). The internal mechanism is hidden; you only care about what goes in and what comes out. In technical terms, a function is a reusable block of code that takes input, performs a task, and optionally returns a result.

Defining a Function: In Python, you define a function using the `def` keyword, followed by the function name, parentheses containing any **parameters**, and a colon. The indented code block beneath it is the function body.

```
def greet(name):
    print(f"Hello,{name}. Welcome to the system.")
```

To use (or **call**) this function:

```
greet("Alice")    # Hello, Alice. Welcome to the system.
greet("Bob")      # Hello, Bob. Welcome to the system.
```

We defined the function once, but we can call it as many times as we want with different arguments.

Parameters vs. Arguments: A **parameter** is the variable name in the function definition (`name` in `def greet(name)`). An **argument** is the actual value you pass in when you call the function (`"Alice"` in `greet("Alice")`). In everyday conversation, people often use these terms interchangeably, but the distinction matters when reading documentation.

Return Values: Functions can send a result back to the caller using the `return` keyword. This is what makes functions truly powerful: they can compute something and hand it back for you to store, print, or use in further calculations.

```
def check_length(password, min_length):
    if len(password) >= min_length:
        return True
    else:
        return False

result = check_length("Tr0ub4dor", 8)
print(result)   # True
```

When Python encounters `return`, it immediately exits the function and sends the specified value back. If a function has no `return` statement, it returns `None` by default.

Multiple Parameters: Functions can accept any number of parameters, separated by commas:

```
def score_password(password, common_list):
    score = 0

    if len(password) >= 8:
        score += 1
    if len(password) >= 12:
        score += 1
    if any(c.isdigit() for c in password):
        score += 1
    if any(c.isupper() for c in password):
        score += 1
    if password not in common_list:
        score += 1

    return score
```

This function takes a password and a list of common passwords, then returns a numeric score. Notice how it uses `len()`, `.isdigit()`, `.isupper()`, and the `in` operator from Core Concepts. We will refine and use this exact function in our capstone task.

Default Parameter Values: You can give a parameter a default value so that the caller does not have to supply it every time:

```
def check_length(password, min_length=8):
    return len(password) >= min_length

print(check_length("short"))          # False   (uses default min_length of 8)
print(check_length("short", 4))       # True    (overrides default with 4)
```

Scope: Variables created inside a function exist only within that function. They are not accessible from the outside. This concept is called **scope**.

```
def calculate():
    result = 42     # local variable
    return result
```

```
calculate()
# print(result)    # This would cause an error: result is not defined here
```

Scope prevents functions from accidentally interfering with each other's data. It is a feature, not a limitation.

Open `functions_demo.py` on the attached VM and run it. The script defines `greet()`, `check_length()`, and `score_password()`. It calls each one and prints the results. After running it, try creating your own function called `is_long_enough(password)` that returns `True` if the password is 12 or more characters.

Answer the questions below

> What keyword sends a value back from a function to the caller?

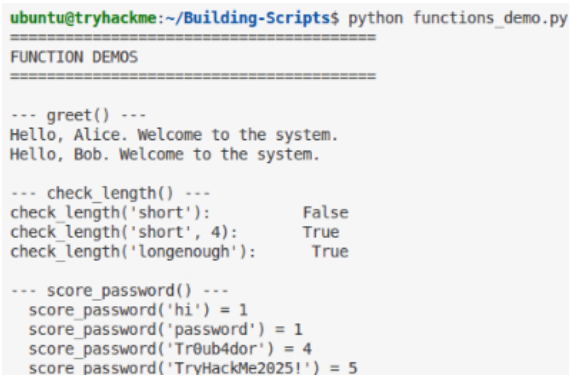
return

> If a function is defined as `def scan(target, port=80):`, what value does `port` take if you call `scan("192.168.1.1")` without specifying a port?

80

> On the VM, run `functions_demo.py`. What score does `score_password` return for the password `"TryHackMe2025!"`?

5



Task 3 - Error Handling

What happens when a user types `"abc"` at a prompt that expects a number? In Simple Demo, our "Guess the Number" game would crash with a `ValueError` because `int("abc")` is not a valid conversion. In a polished program, we do not want a crash; we want a helpful message and a chance to try again. This is where **error handling** comes in.

The Problem: Consider this short script:

```
text = input("Enter a number: ")   # user types "hello"
number = int(text)                  # CRASH: ValueError
print(f"You entered{number}")
```

An example of running this script could be:

```
Enter a number: hello
Traceback (most recent call last):
  File "demo.py", line 2, in <module>
    number = int(text)
ValueError: invalid literal for int() with base 10: 'hello'
```

The program stops entirely. Line 3 never runs. In a security tool that is scanning thousands of hosts, a single unexpected input could halt the entire operation.

The `try` / `except` Block: Python's `try` / `except` structure lets you *attempt* a risky operation and *catch* the error if it occurs, rather than letting the program crash, the program would ask the user to try again:

```
try:
    text = input("Enter a number: ")
    number = int(text)
    print(f"You entered{number}")
except ValueError:
    print("That is not a valid number. Please try again.")
```

An example of running this script could be:

```
Enter a number: hello
That is not a valid number. Please try again.
```

The code inside `try` runs first. If no error occurs, Python skips the `except` block entirely. If an error *does* occur, Python jumps to the matching `except` block and runs that code instead of crashing.

Common Exception Types

Exception	When It Occurs	Example
`ValueError`	Invalid type conversion	`int("abc")`
`FileNotFoundError`	File does not exist	`open("missing.txt")`
`ZeroDivisionError`	Division by zero	`10 / 0`
`KeyError`	Dictionary key not found	`d["missing_key"]`
`IndexError`	List index out of range	`mylist[99]`

Catching Multiple Exceptions: You can handle different errors differently:

```
try:
    filename = input("File to open: ")
    with open(filename) as f:
        data = f.read()
    port = int(data.strip())
except FileNotFoundError:
    print(f"Error: '{filename}' does not exist.")
except ValueError:
    print(f"Error: the file does not contain a valid number.")
```

The Generic `except` Using `except Exception as e` catches most exceptions. Use it sparingly, as it can hide bugs you would want to know about:

```
try:
    risky_operation()
except Exception as e:
    print(f"Something went wrong:{e}")
```

The `as e` part stores the exception's message in the variable `e`, which you can print for debugging.

Combining Error Handling with Loops: A common and powerful pattern is to wrap user input in a loop so the program keeps asking until valid input is provided:

```
while True:
    try:
        age = int(input("Enter your age: "))
        break   # valid input received; exit the loop
    except ValueError:
        print("Invalid input. Please enter a whole number.")
```

This pattern is essential in interactive tools. Our Password Strength Checker will use it to ensure the user enters a non-empty password before processing.

On the VM, open `error_demo.py` and run it. The script deliberately triggers several types of exceptions and catches each one. Study how each `try` / `except` block handles its specific error type.

Answer the questions below

> What type of exception is raised when you attempt `int("hello")`?

ValueError

> What type of exception is raised when you try to open a file that does not exist?

FileNotFoundError

> On the VM, open a Python terminal and type `print(10 / 0)`. What is the name of the exception that Python raises?

ZeroDivisionError

Task 4 - Reading and Writing Files

In cyber security, you will constantly work with files: reading wordlists for brute-force attacks, parsing log files for suspicious activity, or writing scan results to a report. Python makes file operations straightforward, and modern best practices ensure they are done safely.

Reading a File: The built-in `open()` function opens a file. You pass it the file path and a **mode**: `"r"` for reading, `"w"` for writing, and `"a"` for appending.

```
f = open("passwords.txt", "r")
content = f.read()
```

```
print(content)
f.close()
```

This works, but there is a problem. If an error occurs between `open()` and `close()`, the file stays open in memory. Over time, leaked file handles can cause performance issues or data corruption. Python solves this with **context managers**.

The `with` Statement (Context Managers): The recommended way to work with files in modern Python is the `with` statement. It automatically closes the file when the indented block ends, even if an error occurs:

```
with open("passwords.txt", "r") as f:
    content = f.read()

print(content)   # the file is already closed at this point
```

From this point forward, always use `with` for file operations. It is safer, cleaner, and considered a best practice across the industry.

Reading Methods

Method	Returns	Best For
`.read()`	Entire file as a single string	Small files you process as one block
`.readline()`	The next single line	Processing one line at a time
`.readlines()`	A list where each element is one line	When you need all lines as a list

A common and memory-efficient pattern is to loop directly over the file object:

```
with open("common_passwords.txt", "r") as f:
    for line in f:
        password = line.strip()    # remove the trailing newline
        print(password)
```

The `.strip()` call is essential here. Each line in a text file ends with a newline character (`\n`). Without `.strip()`, you would get `"password123\n"` instead of `"password123"`.

Loading a Wordlist into a List: For our Password Strength Checker, we want to load a file of common passwords into a Python list so we can check user input against it:

```
common_passwords = []

with open("common_passwords.txt", "r") as f:
    for line in f:
        common_passwords.append(line.strip())

print(f"Loaded{len(common_passwords)} common passwords.")
```

Now we can use the `in` operator to check whether a password is weak:

```
if user_password in common_passwords:
    print("This password appears in the common-passwords list.")
```

Writing to a File: To create a new file or overwrite an existing one, use mode `"w"`:

```
with open("results.txt", "w") as f:
    f.write("Scan Results\n")
    f.write("============\n")
    f.write(f"Target: 192.168.1.1\n")
    f.write(f"Open ports: 22, 80, 443\n")
```

To add content to the end of an existing file without erasing it, use mode `"a"` (append):

```
with open("results.txt", "a") as f:
    f.write(f"Additional finding: port 3306 open\n")
```

Common File Modes

Mode	Description
`"r"`	Read (file must exist)
`"w"`	Write (creates file or *overwrites* existing content)
`"a"`	Append (creates file or adds to existing content)

Be careful with `"w"` mode. If the file already exists, opening it in write mode will erase all its contents immediately. If you want to preserve the original content and add to it, always use `"a"`.

On the VM, a file called `common_passwords.txt` is saved in the `/home/ubuntu/Building-Scripts/` directory. Open `files_demo.py` and run it. The script loads the wordlist, prints how many passwords it found, and checks a sample password against the list. Then, open `common_passwords.txt` itself in VS Code and note how many lines it contains.

Answer the questions below

> What keyword introduces a context manager for safely opening files?

with

> What string method removes the trailing newline character from each line read from a file?

.strip()

> On the VM, run `files_demo.py`. How many passwords did it load from `common_passwords.txt`?

58

Task 5 - Libraries and Pip

In Simple Demo, we used `import random` to access the `random.randint()` function. That single line enabled us to generate random numbers without writing the algorithm ourselves. This concept of importing pre-written code is one of Python's greatest strengths.

A **library** (also called a **module** or **package** depending on context) is a collection of pre-written code that you can import into your own programs. One analogy would be a toolbox: instead of forging a wrench from raw metal every time you need one, you reach into the toolbox and grab the wrench someone already made. In technical terms, libraries save you from reinventing the wheel.

Importing Libraries - Python provides several ways to import:

```python
# Import the entire library
import os
print(os.getcwd())          # prints the current working directory

# Import a specific function
from datetime import datetime
now = datetime.now()
print(f"Current time:{now}")

# Import with an alias (nickname)
import datetime as dt
now = dt.datetime.now()
```

The Python Standard Library: Python ships with a large collection of built-in modules called the **standard library**. These are available out of the box without installing anything. Here are a few you will encounter often:

Module	Purpose	Example Use
`os`	Interact with the operating system	List files, get environment variables
`sys`	System-specific parameters	Read command-line arguments
`random`	Generate random numbers	Pick random items, shuffle lists
`datetime`	Work with dates and times	Timestamps, time calculations
`json`	Read and write JSON data	Parse API responses, config files
`hashlib`	Cryptographic hashing	MD5, SHA-256 hashes
`string`	String constants and utilities	Access `string.punctuation`, `string.digits`

For our Password Strength Checker, the `string` module is particularly useful. It provides constants like `string.ascii_uppercase`, `string.digits`, and `string.punctuation` that we can use to check password character variety without hardcoding every special character ourselves:

```python
import string

password = "S3cure!Pass"

has_upper = any(c in string.ascii_uppercase for c in password)
has_digit = any(c in string.digits for c in password)
has_special = any(c in string.punctuation for c in password)

print(f"Uppercase:{has_upper}")     # True
print(f"Digit:{has_digit}")         # True
print(f"Special:{has_special}")     # True
```

Installing Third-Party Libraries with `pip` Beyond the standard library, thousands of community-built libraries are available through the **Python Package Index (PyPI)**. You install them using `pip`, Python's package manager, from the terminal:

```
pip install requests
```

Once installed, you import them just like any standard library module:

```
import requests

response = requests.get("https://tryhackme.com")
print(response.status_code)   # 200
```

Security-Relevant Libraries

- **requests**: send HTTP requests (web scraping, API interaction)
- **scapy**: craft, send, and sniff network packets
- **pwntools**: CTF and binary exploitation toolkit
- **paramiko**: SSH client and server implementation
- **beautifulsoup4**: parse and extract data from HTML pages

You do not need to memorize these now. The key takeaway is that Python's ecosystem lets you go from "I need to send a packet" to "the packet is sent" in just a few lines of code, because someone else already built the hard parts.

On the VM, open `imports_demo.py` and run it. The script demonstrates imports from `os`, `datetime`, `string`, and `hashlib`. After running it, observe how `hashlib` computes a SHA-256 hash. Try changing the input string and rerunning to see how the hash changes completely.

Answer the questions below

> What is the name of Python's package manager used to install third-party libraries?

pip

> Which module from the standard library provides constants like `ascii_uppercase`, `digits`, and `punctuation`?

string

> On the VM, run `imports_demo.py`. What is the first character of the SHA-256 hash it prints for the default input string?

5

Task 6 - Putting It All Together: Password Strength Checker

Over the previous tasks and the Core Concepts room, we learned about data types, strings, lists, dictionaries, operators, loops, functions, error handling, file I/O, and libraries. Now it is time to combine all those concepts into a single, working program: a **Password Strength Checker**.

Our program will do the following:

1. Load a list of common passwords from a file (`common_passwords.txt`)
2. Prompt the user for a password
3. Check the password's length, character variety, and presence in the common list
4. Assign a strength score and a label (Weak, Moderate, or Strong)
5. Display the result and write it to a log file

Let's build it step by step. The complete program is saved as `password_checker.py` in the `/home/ubuntu/Building-Scripts/` directory on the attached VM.

Step 1: Imports and Common Passwords

```
import string

def load_common_passwords(filepath):
    """Load a list of common passwords from a text file."""
    common = []
    try:
        with open(filepath, "r") as f:
            for line in f:
                common.append(line.strip().lower())
    except FileNotFoundError:
        print(f"Warning: '{filepath}' not found. Skipping common-password check.")
    return common
```

This function uses file I/O (Task 4), error handling (Task 3), lists and loops (Core Concepts), and a function definition (Task 2). Notice the `try`/`except` block: if the file does not exist, the program prints a warning and continues instead of crashing. The `.lower()` call normalises each password, making comparisons case-insensitive.

Step 2: The Scoring Function

```
def check_password(password, common_list):
    """Evaluate a password and return (score, feedback_list)."""
    score = 0
    feedback = []

    # Length checks
    if len(password) >= 8:
        score += 1
    else:
        feedback.append("Password should be at least 8 characters.")

    if len(password) >= 12:
        score += 1

    # Character variety checks
    if any(c in string.ascii_uppercase for c in password):
        score += 1
    else:
        feedback.append("Add at least one uppercase letter.")

    if any(c in string.digits for c in password):
        score += 1
    else:
        feedback.append("Add at least one digit.")
```

```
    if any(c in string.punctuation for c in password):
        score += 1
    else:
        feedback.append("Add at least one special character (e.g., !, @, #).")

    # Common password check (overrides all other scoring)
    if password.lower() in common_list:
        score = 0
        feedback = ["This password is in the common-passwords list. Choose anothe
r."]

    return score, feedback
```

This function uses strings and string methods, the `in` operator, conditional logic, lists, and the `string` module (Task 5). It returns *two* values: a numeric score and a list of suggestions. In Python, you can return multiple values separated by commas; the caller receives them as a **tuple**.

Step 3: The Main Program

```
def main():
    strength_labels = {
        0: "Weak", 1: "Weak",
        2: "Moderate", 3: "Moderate",
        4: "Strong", 5: "Strong"
    }

    common_list = load_common_passwords("common_passwords.txt")

    while True:
        password = input("\nEnter a password to check (or 'quit' to exit): ")

        if password.lower() == "quit":
            print("Goodbye.")
            break

        if len(password) == 0:
            print("Password cannot be empty. Try again.")
            continue

        score, feedback = check_password(password, common_list)
        label = strength_labels.get(score, "Unknown")

        print(f"\nStrength:{label} ({score}/5)")

        if feedback:
            print("Suggestions:")
            for tip in feedback:
                print(f"  -{tip}")
```

```
        # Log the result (mask the actual password with asterisks)
        with open("password_log.txt", "a") as log:
            log.write(f"Password:{'*' * len(password)} | Strength:{label} ({score}/
5)\n")

main()
```

This final section uses a dictionary for label mapping, a `while True` loop with `break` and `continue`, f-strings, file writing in append mode (Task 4), and function calls (Task 2). Notice that we do not log the actual password; we write asterisks instead. In a security context, never store plaintext passwords in logs.

Running the Program

```
$ python3 password_checker.py

Enter a password to check (or 'quit' to exit): password

Strength: Weak (0/5)
Suggestions:
  - This password is in the common-passwords list. Choose another.

Enter a password to check (or 'quit' to exit): Tr0ub4dor

Strength: Moderate (3/5)
Suggestions:
  - Add at least one special character (e.g., !, @, #).

Enter a password to check (or 'quit' to exit): C0mpl3x!Pass#99

Strength: Strong (5/5)

Enter a password to check (or 'quit' to exit): quit
Goodbye.
```

Every concept from both rooms is present in this program. We encourage you to open `password_checker.py` on the VM, run it, study the code, and experiment. Try adding new checks (minimum number of special characters, detection of consecutive repeated characters, dictionary word rejection) to make the checker even more robust.

Answer the questions below

> In the `check_password` function, what score does the password receive if it is found in the common_passwords list?

0

> On the VM, run `password_checker.py` and enter the password `TryHackMe!2025`. What strength label does the program display?

Strong

> After running the checker with the password `TryHackMe!2025`, open `password_log.txt` on the VM. How many asterisks appear on that line?

14

Task 7 - Conclusion

In this room, we covered the capabilities that turn code snippets into real scripts:

- **Functions**: defining reusable blocks of code with `def`, parameters, return values, defaults, and scope
- **Error handling**: using `try` / `except` to catch exceptions and build resilient programs that do not crash on unexpected input
- **File I/O**: reading from and writing to files safely with the `with` context manager, and understanding `"r"`, `"w"`, and `"a"` modes
- **Libraries**: importing standard modules like `os`, `string`, and `hashlib`, and installing third-party packages with `pip`